PROJECT GUIDE · PART 1

# AskMyDocs Fresher Guide

Project kya karta hai, kaise chalta hai, aur andar data kaise behta hai: shuru se, simple Hinglish me

| | |
|---|---|
| **Project** | AskMyDocs: apne documents se baat karne wala AI assistant |
| **Stack** | .NET 10 Web API · Angular 21 · Microsoft Agent Framework · Vector DB |
| **AI setup (abhi)** | Ollama (local, free): `llama3.2` + `nomic-embed-text` |
| **Vector store (abhi)** | InMemory (RAM me) |
| **Date** | 15 October 2026 |
| **Saath wala document** | Part 2: AskMyDocs Technical Documentation |

## Is guide me kya hai

1. Project ek nazar me: kya hai, kyun hai, real life me kahan use hota hai
2. Project me kitne projects hain aur kaise chalana hai
3. Basic concepts: LLM, embedding, chunk, vector DB, RAG, kernel, agent
4. Architecture: kaun kisse baat karta hai
5. Data flow 1: document upload (indexing)
6. Data flow 2: sawaal ka jawab (Plain RAG)
7. Data flow 3: Agent aur streaming
8. Data kahan save hota hai
9. Semantic Kernel vs Agent Framework
10. Har module ki summary
11. Common errors aur unka hal
12. Seekhne ka sahi order

> **Is guide ko kaise padhein**
> Pehle section 1 aur 2 padho, taaki project chal jaye. Phir section 3 ke concepts samjho. Uske baad section 5, 6 aur 7 (data flow) padho. Concept samajh aate hi code kholkar wahi cheez dhoondo. Har concept ke neeche likha hai ki wo kis file me hai.

# 1. Project ek nazar me

## 1.1 Ye project kya karta hai?

**AskMyDocs ek AI assistant hai jo aapke documents padhkar unke baare me sawaal ka jawab deta hai.** Aap ek document upload karte ho (jaise company ki IT policy), phir normal bhasha me sawaal poochte ho. Assistant document me se sahi hissa dhoondhkar jawab deta hai aur saath me batata hai ki jawab document ke kis hisse se aaya.

> **Ek example**
>
> Aapne `sample-docs/itsm-policy.md` upload kiya. Usme likha hai: *"P1 (Critical): resolution SLA 4 ghante."*
>
> Aap poochte ho: **"P1 ticket ka SLA kitna hai?"**
>
> Jawab aata hai: **"P1 ticket ka resolution SLA 4 ghante hai [1]."** Saath me ek box dikhta hai: *[1] itsm-policy.md, chunk 0, score 0.71* aur us chunk ka text. Yaani aap khud check kar sakte ho ki jawab sahi jagah se aaya.
>
> (Score jaisi exact values aapke model aur document par depend karti hain. Ye sirf samjhane ke liye hai.)

## 1.2 ChatGPT se seedha kyun nahi pooch lete?

Kyunki ChatGPT (ya koi bhi LLM) ko **aapki company ki policy pata hi nahi hai**. Wo internet ke public data par train hua hai. Aapki internal IT policy, HR rules ya product manual uski training me nahi the. Aise me LLM ya to mana kar dega, ya (jo zyada khatarnak hai) **confident hokar koi galat number bata dega**. Isko **hallucination** kehte hain.

AskMyDocs ye problem ek simple trick se solve karta hai:

1. Pehle aapke document me se sawaal se juda hissa **dhoondo**.
2. Wo hissa LLM ko do aur bolo: **"Sirf isi text se jawab do, apni taraf se kuch mat jodo."**

Isi technique ko **RAG (Retrieval-Augmented Generation)** kehte hain. Ye open-book exam jaisa hai: student (LLM) ko sab kuch yaad nahi, lekin sahi page khol diya jaye to wo sahi jawab likh deta hai.

## 1.3 Sirf jawab nahi, kaam bhi: Agent

Project ka doosra hissa **Agent** hai. Agent sirf documents se jawab nahi deta, wo **khud decide karta hai ki kaunsa tool chalana hai**. Is project me agent ke paas 3 tools hain:

- `search_documents` : uploaded documents me search karna
- `get_ticket_status` : kisi IT ticket (jaise INC-1001) ka status batana
- `create_ticket` : naya ticket banana

Example: *"INC-1001 ka status batao aur uski priority ka SLA bhi."* Agent pehle ticket tool chalata hai, jisse pata chalta hai ki priority P2 hai. Phir document search karke P2 ka SLA (8 ghante) nikalta hai. Ek sawaal me do tool, aur sab apne aap.

## 1.4 Real life me kahan use hota hai?

| Use case | Kaise |
|---|---|

| | |
|---|---|
| **IT Helpdesk / ITSM bot** | Employees policy, SLA, VPN access, password reset ke baare me poochte hain. Bot ticket status bhi batata hai aur ticket bhi banata hai. Ye project isi domain par bana hai. |
| **HR policy assistant** | "Kitni leave carry forward hoti hai?", "Reimbursement ka process kya hai?" HR ko roz same sawaal ka jawab nahi dena padta. |
| **Customer support** | Product manual ya FAQ upload karo. Customer ke sawaal ka jawab manual ke reference ke saath milta hai. |
| **Legal / compliance** | Lambe contracts aur policies me se specific clause dhoondhna. |
| **Internal wiki / developer docs** | "Deploy kaise karte hain?", "Ye API kya leti hai?" Naye developer ka onboarding fast hota hai. |
| **Sales / pre-sales** | Product brochures aur RFP answers me se turant sahi jawab. |

> **Ye ek learning project bhi hai**
>
> README ke hisaab se ye `bhagat_ainotes.pdf` book ke saath chalne wala project hai. Isliye isme ek **Labs** project bhi hai, jisme har concept (embedding, vector DB, chunking, RAG, Semantic Kernel, agent, multi-agent) ka alag chhota demo hai. Code ke comments bhi Hinglish me samjhaate hain.

# 2. Project me kitne projects hain aur kaise chalana hai

Folder `AskMyDocs` me **3 code projects** hain. Saath me **2 bahar ki services** hain jinke upar ye projects chalte hain.

## 2.1 Teen code projects

| # | Project | Folder | Technology | Kaam | Port |
|---|---|---|---|---|---|
| 1 | **AskMyDocs.Api** (main project) | `api/AskMyDocs.Api` | .NET 10 Web API (C#) | Backend: document upload, chunking, embedding, search, RAG jawab, agent, streaming | 5080 |
| 2 | **askmydocs-web** | `web/askmydocs-web` | Angular 21 (TypeScript) | Chat screen (UI). Iska apna AI logic nahi hai, ye sirf API ko call karta hai. | 4200 |
| 3 | **AskMyDocs.Labs** | `labs/AskMyDocs.Labs` | .NET 10 console app | 7 chhote seekhne wale demos. Main app ka hissa nahi hai. | - |

## 2.2 Bahar ki services (projects nahi, par zaroori)

| Service | Kaam | Zaroori? | Port |
|---|---|---|---|
| **Ollama** | AI model aapke computer par chalata hai. Chat ( `llama3.2` ) aur embedding ( `nomic-embed-text` ) dono yahi deta hai. Free hai, key nahi chahiye. | Haan (jab tak OpenAI key nahi hai) | 11434 |
| **Qdrant** | Asli vector database. Data disk par save rehta hai. `docker-compose.yml` se chalta hai. | Nahi (optional). Abhi InMemory use ho raha hai. | 6333, 6334 |

## 2.3 Baaki files

- `AskMyDocs.slnx` : Visual Studio solution file. Isme API aur Labs hain. Angular isme nahi hai, wo alag chalta hai.
- `sample-docs/itsm-policy.md` : test karne ke liye ek sample IT policy document.
- `docker-compose.yml` : Qdrant chalane ki file.
- `README.md` : project ki original guide.

## 2.4 Chalane ka poora tareeka

### Ek baar ka setup (ho chuka hai)

```
# Ollama install karo (ollama.com/download), phir models download karo:
ollama pull llama3.2
ollama pull nomic-embed-text

# Angular ki libraries (pehli baar):
cd C:\Users\bhagat.singh\Desktop\AskMyDocs\web\askmydocs-web
npm install
```

### Har baar chalane ke steps

1. **Ollama chal raha hai, ye check karo.** Browser me `http://localhost:11434` kholo. "Ollama is running" dikhna chahiye. Windows par install ke baad ye apne aap background me chalta hai.

2. **Terminal 1: API chalao**

```
cd C:\Users\bhagat.singh\Desktop\AskMyDocs\api\AskMyDocs.Api
dotnet run
```

   Jab `Now listening on: http://localhost:5080` dikhe, browser me `http://localhost:5080/health` kholo. Waha `"chatModel":"llama3.2"` dikhna chahiye.

3. **Terminal 2: Angular UI chalao**

```
cd C:\Users\bhagat.singh\Desktop\AskMyDocs\web\askmydocs-web
npm start
```

   `Local: http://localhost:4200/` dikhe to browser me kholo.

4. **Use karo.** Left side se `.txt` / `.md` file upload karo, mode chuno (Plain RAG / Agent / Agent + streaming), aur sawaal poocho.

5. **Band karna.** Dono terminals me **Ctrl+C** dabao.

### Labs chalana (seekhne ke liye)

```
cd C:\Users\bhagat.singh\Desktop\AskMyDocs\labs\AskMyDocs.Labs
dotnet run -- lab3    # chunking (AI ki zaroorat nahi, yahin se shuru karo)
dotnet run -- lab1    # embedding aur cosine similarity
dotnet run -- lab2    # vector store: save, search, filter
dotnet run -- lab4    # mini RAG: bina RAG vs RAG ke saath
dotnet run -- lab5    # Semantic Kernel
dotnet run -- lab6    # Agent Framework: agent + tools + session
dotnet run -- lab7    # multi-agent workflow
```

### Qdrant (optional, Docker chahiye)

```
cd C:\Users\bhagat.singh\Desktop\AskMyDocs
docker compose up -d
# phir api/AskMyDocs.Api/appsettings.json me:  "VectorStore": { "Kind": "Qdrant" }
# API restart karo. Dashboard: http://localhost:6333/dashboard
```

> **Dhyan rakhne wali baatein**
>
> - `dotnet run` sirf **API folder** (ya Labs folder) me chalta hai. Angular folder me `npm start` chalta hai.
> - API restart karni ho to **usi terminal** me Ctrl+C dabao jisme wo chal rahi hai. Doosre terminal me dobara `dotnet run` chalaoge to "file is locked" error aayega.
> - Pehle API chalni chahiye, phir UI. UI khud kuch nahi karta, wo API se hi baat karta hai.
> - Visual Studio me: `AskMyDocs.slnx` kholo, **AskMyDocs.Api** par right-click karke *Set as Startup Project* chuno, phir F5 dabao.

## 3. Basic concepts (fresher ke liye)

Ye concepts samajh aa gaye to poora project samajh aa jayega. Har concept ke neeche likha hai ki wo project me kahan use hota hai.

### LLM (Large Language Model)

Ek bahut bada AI model jo text padhkar aage ka text likhta hai. ChatGPT, Llama, Gemini sab LLM hain. Ye "sochta" nahi, balki bahut achhe se andaza lagata hai ki aage kya likha jana chahiye.

**Project me:** `llama3.2` (Ollama se). Code me ise `IChatClient` interface se call kiya jata hai (`Program.cs`).

### Prompt aur System Prompt

**Prompt** = LLM ko bheja gaya text. **System prompt** = LLM ke liye niyam, jaise "Tum ek document assistant ho, sirf CONTEXT se jawab do." LLM har jawab me in niyamon ko follow karne ki koshish karta hai.

**Project me:** `RagService.cs` me `SystemPrompt`, aur `SupportAgentFactory.cs` me agent ke `Instructions`.

### Token

LLM text ko chhote tukdon (tokens) me padhta hai. Ek token lagbhag 3-4 English characters ke barabar hota hai. LLM ki limit (kitna padh sakta hai) aur cost (OpenAI me) tokens me hi gini jaati hai.

**Project me:** `MaxOutputTokens = 800`, yaani jawab 800 tokens se lamba nahi hoga (`RagService.cs`).

### Embedding (sabse zaroori concept)

Embedding = **text ko numbers ki ek lambi list me badalna**. `nomic-embed-text` har text ke liye 768 numbers banata hai, jaise `[0.021, -0.113, 0.087, ...]`.

Khaas baat ye hai ki **jin texts ka matlab milta-julta hai, unke numbers bhi paas-paas hote hain**, chahe shabd alag hon. "Mera laptop start nahi ho raha" aur "Laptop boot nahi ho raha, screen kaali hai" ke vectors bahut paas honge. "Aaj Delhi me barish ho rahi hai" door hoga.

*Socho har text ka map par ek location hai. Milte-julte matlab wale texts ek hi mohalle me rehte hain.*

**Project me:** `IEmbeddingGenerator` (`Program.cs`). Labs: `Lab01_Embeddings.cs`.

### Vector aur Cosine Similarity

Numbers ki wo list hi **vector** hai. Do vectors kitne milte hain, ye **cosine similarity** se naapte hain. Ye do vectors ke beech ka angle hai: **1 = bilkul same matlab**, **0 = koi lena-dena nahi**.

```
cos(A, B) = (A . B) / (|A| * |B|)
```

**Project me:** `DocChunk.cs` me `DistanceFunction.CosineSimilarity`. Formula haath se: `AiConfig.CosineSimilarity()` (Labs).

### Chunking

Bade document ko chhote tukdon (**chunks**) me todna. Is project me har chunk lagbhag **1200 characters** ka hota hai, aur pichle chunk ke aakhri **200 characters** agle chunk me dobara aate hain (**overlap**). Isse koi sentence do chunks ke beech me kat kar gayab nahi hota.

**Kyun?** Poore 50 page ka ek hi vector banaoge to wo "average matlab" ban jayega aur kisi specific sawaal se match nahi karega. Chhote chunks se sahi paragraph milta hai, aur LLM ko kam text bhejna padta hai.

**Project me:** `Rag/TextChunker.cs` . Labs: `Lab03_Chunking.cs` .

### Vector Database (Vector Store)

Ek database jo vectors save karta hai aur ek kaam bahut fast karta hai: **"Is sawaal ke vector ke sabse paas wale 4 vectors do."** Normal SQL `LIKE '%laptop%'` sirf shabd match karta hai. Vector DB **matlab** match karta hai.

**Project me:** do options. **InMemory** (RAM me, abhi yahi chal raha hai) aur **Qdrant** (asli DB, disk par). Dono ka code same hai, sirf config badalti hai.

### RAG (Retrieval-Augmented Generation)

Teen step ka tareeka:

- **R - Retrieve:** sawaal ka embedding banao aur vector DB me sabse milte-julte chunks dhoondo.
- **A - Augment:** un chunks ko prompt me "CONTEXT" ke roop me chipka do.
- **G - Generate:** LLM se bolo "sirf is CONTEXT se jawab do, aur source [1], [2] likho."

*Open-book exam: LLM ko sab yaad nahi, par sahi page khol do to sahi jawab likh deta hai.*

**Project me:** `Rag/RagService.cs` . Labs: `Lab04_MiniRag.cs` .

### Agent

**Agent = LLM + instructions + tools + memory.** Normal chat me aap message bhejte ho aur text aata hai, bas. Agent **khud decide karta hai** ki kaunsa tool chalana hai, kitni baar, aur kab rukna hai.

*Ek office assistant socho jiske paas file cabinet (search_documents), ticket register (get_ticket_status) aur naya ticket form (create_ticket) hai. Aap sawaal poochte ho, wo khud tay karta hai ki kaunsi cheez kholni hai.*

**Project me:** `Agents/SupportAgentFactory.cs` , `Agents/SupportTools.cs` . Labs: `Lab06_Agent.cs` .

### Tool / Function Calling

LLM khud C# code nahi chala sakta. Wo sirf bolta hai: "*mujhe `get_ticket_status` ko `INC-1001` ke saath chalana hai.*" Hamara code (library) wo C# method chalata hai, result LLM ko wapas deta hai, aur LLM final jawab likhta hai. LLM method ka **naam aur** `[Description]` padhkar hi tay karta hai ki kab kaunsa tool chalana hai.

**Project me:** `UseFunctionInvocation()` ( `Program.cs` ) ye loop chalata hai. `AIFunctionFactory.Create(...)` C# method ko tool banata hai ( `SupportTools.cs` ).

### Session (memory)

LLM ko pichli baat yaad nahi rehti. Har baar poori pichli conversation bhejni padti hai. **Session** wahi pichli conversation sambhal ke rakhta hai. Isliye aap pooch sakte ho "aur uska owner kaun hai?" aur agent samajh jata hai ki "uska" matlab INC-1001.

**Project me:** `SupportAgentFactory` ki `_sessions` dictionary. UI har request me `sessionId` bhejta hai.

### Streaming (SSE: Server-Sent Events)

Poora jawab banne ka intezaar karne ki jagah, jawab **shabd-dar-shabd** (token by token) screen par aata hai, ChatGPT jaisa. Server ek hi HTTP response me thoda-thoda data bhejta rehta hai: `data: {...}`.

**Project me:** `/api/agent/stream` endpoint ( `Program.cs` ). UI me `streamAgent()` ( `askmydocs.service.ts` ).

### Semantic Kernel (Kernel)

Microsoft ka AI SDK. **Kernel** ek container hai jisme AI service (LLM) aur aapke C# functions (**plugins**) ek jagah rakhe jaate hain. Kernel LLM ko batata hai: "ye functions available hain, zaroorat ho to bula lena."

**Project me: sirf Lab 5 me** ( `Lab05_SemanticKernel.cs` ). Main API me Semantic Kernel use **nahi** hota. Wahan iska naya version (Agent Framework) use hota hai. Section 9 me detail hai.

### Microsoft.Extensions.AI aur Microsoft.Extensions.VectorData

Ye do "common interface" libraries hain. `IChatClient` aur `IEmbeddingGenerator` kisi bhi AI provider (OpenAI, Ollama, Azure) ke liye same hain. `VectorStoreCollection` kisi bhi vector DB (InMemory, Qdrant) ke liye same hai. **Fayda:** provider ya database badlo, code nahi badalta, sirf config badalti hai.

# 4. Architecture: kaun kisse baat karta hai

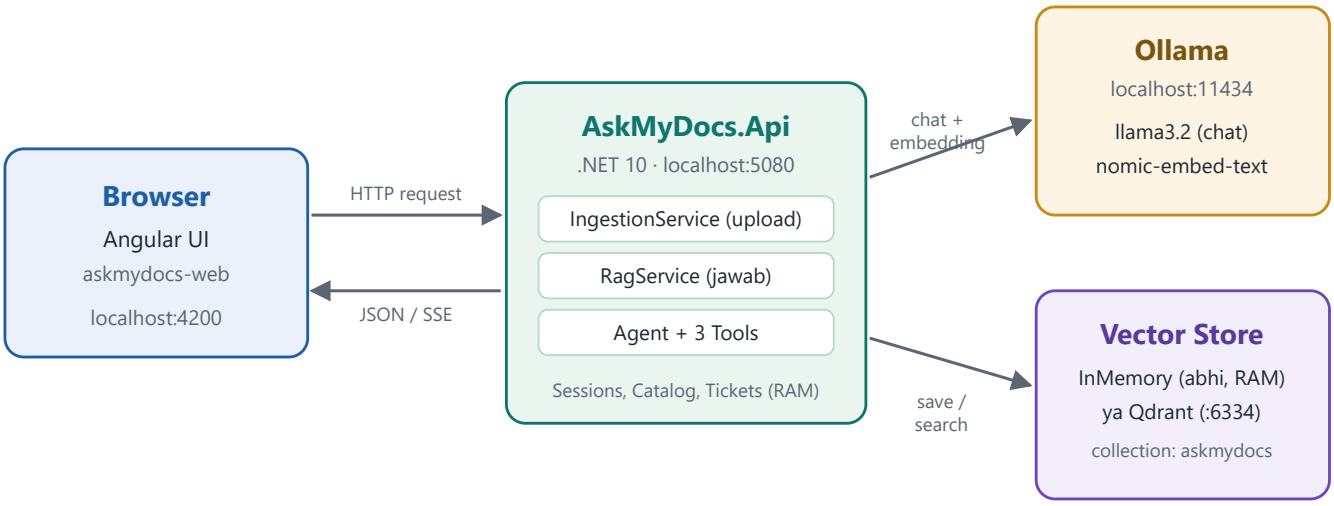



Figure 1: AskMyDocs ke 4 hisse aur unke beech ka connection

- **Browser (Angular UI)** sirf screen hai. Wo API ko HTTP request bhejta hai aur jawab dikhata hai.
- **API** dimaag hai. Saara logic yahan hai: document todna, embedding banwana, search karna, LLM se jawab likhwana, agent chalana.
- **Ollama** AI ka engine hai. API isse do kaam karwati hai: **embedding banana** (text se numbers) aur **jawab likhna** (chat).
- **Vector Store** godown hai. Chunks aur unke vectors yahan rakhe jaate hain, aur sawaal aane par yahin search hoti hai.

> **Provider badalna = sirf config**
>
> API OpenAI ke format me baat karti hai. Ollama bhi wahi format samajhta hai (`http://localhost:11434/v1`). Isliye `appsettings.json` me Endpoint badal do to wahi code OpenAI, Azure OpenAI ya Ollama, kisi ke saath bhi chal jata hai.

## 5. Data flow 1: Document upload (Indexing)

Ye kaam **sawaal poochne se pehle** hota hai. Document ko aisi shakal me rakhna jisse baad me search ho sake, isi ko **indexing** kehte hain.

File (.md/.txt) → Text → Chunks → Embeddings (768 numbers) → Vector Store

1. **User file chunta hai** (Angular, `app.ts → onFileSelected()`). Browser `file.text()` se file ka poora text padhta hai. **File server par nahi jaati, sirf uska text jata hai.** Isliye abhi sirf text files chalti hain (`.txt, .md, .json, .csv`). PDF/Word nahi chalti.
2. **API ko request jaati hai** (`askmydocs.service.ts → uploadDocument()`):

```
POST http://localhost:5080/api/documents
{ "fileName": "itsm-policy.md", "content": "# WorkElevate ITSM Policy ... (poora text)" }
```

3. **API ka endpoint** (`Program.cs`) request ko `IngestionService.IngestAsync()` ko de deta hai.
4. **Collection ready karo.** `EnsureCollectionExistsAsync()`: vector store me `askmydocs` naam ka collection (table jaisa) nahi hai to bana do.
5. **Document ko ID do.** `documentId = Guid.NewGuid()`, jaise `152441e486a9473cbecab2cc5858a81f`.
6. **Chunking** (`TextChunker.Chunk()`):
   - Blank line par paragraph todo.
   - Paragraphs jodte jao jab tak ~1200 characters na ho jayein. Phir chunk band karo.
   - Naye chunk ki shuruaat me pichle chunk ke aakhri 200 characters daalo (overlap).
   - Koi akela paragraph 1200 se bada ho to use sentence (`. ! ?`) par todo.
   - 30 characters se chhote chunks phenk do. Wo bas noise hain.
7. **Har chunk ka record banao** (`DocChunk`): `Id`, `DocumentId`, `FileName`, `ChunkIndex` (0, 1, 2...), `Text`, aur `Embedding = text`.
8. **Save karo** (`collection.UpsertAsync(records)`). Yahan jaadu hota hai: collection ke paas embedding generator hai, isliye library **khud** har chunk ka text Ollama (`nomic-embed-text`) ko bhejti hai. Har chunk ke 768 numbers aate hain, aur record + vector store me save ho jata hai. Saare chunks ek saath (batch) bheje jaate hain, taaki fast ho.
9. **Catalog me entry**: document ki summary (`DocumentSummary`: id, file name, kitne chunks, kab upload hua) ek dictionary me rakhi jaati hai. Sidebar ki list yahin se aati hai.
10. **Jawab wapas**: `{ "documentId": "...", "fileName": "itsm-policy.md", "chunkCount": 3 }`. Sample document ke sach me 3 chunks bante hain (1186, 1178 aur 537 characters). UI dikhata hai "*itsm-policy.md index ho gaya - N chunks bane*" aur sidebar refresh karta hai (`GET /api/documents`).

## Ek chunk store me kaisa dikhta hai

```
{
  "Id":         "7f3c...e21a",                       // har chunk ka unique Guid
  "DocumentId": "152441e486a9473cbecab2cc5858a81f",   // kis document ka hai
  "FileName":   "itsm-policy.md",
  "ChunkIndex": 0,                                    // document me kaunsa number
  "Text":       "# WorkElevate ITSM Policy ... ## SLA aur Priority ... P1 (Critical): resolution SLA 4
ghante ...",
  "Embedding":  [0.0213, -0.1134, 0.0871, ... 768 numbers]   // search isi par hoti hai
}
```

# 6. Data flow 2: Sawaal ka jawab (Plain RAG mode)

Sawaal → Sawaal ka embedding → Top 4 chunks (Retrieve) → Prompt me context (Augment) → LLM jawab (Generate) → Jawab + sources

1. **User sawaal bhejta hai** (mode: Plain RAG). UI request bhejta hai: `POST /api/ask { "question": "P1 ticket ka SLA kitna hai?" }`
2. **RagService.AskAsync()** chalta hai.
3. **RETRIEVE** (`RetrieveAsync`): `collection.SearchAsync(question, 4)`.
   - Library sawaal ka bhi embedding banwati hai (Ollama se, 768 numbers).
   - Vector store saare chunks ke vectors se cosine similarity nikalta hai.
   - Sabse zyada score wale **4 chunks** aate hain.
   - Jinka score **0.25 se kam** hai wo hata diye jaate hain, kyunki wo kachra context hai.
4. **Kuch nahi mila?** Agar ek bhi chunk nahi bacha to API seedha bolti hai *"Diye gaye documents me iska jawab nahi mila."* LLM ko call hi nahi kiya jata. Isse time bachta hai aur hallucination ka chance nahi rehta.
5. **AUGMENT** (`BuildContext`): chunks ko numbered context me badlo:

```
[1] (file: itsm-policy.md, chunk: 0)
# WorkElevate ITSM Policy ... ## SLA aur Priority ... P1 (Critical): resolution SLA 4 ghante ...

[2] (file: itsm-policy.md, chunk: 1)
...
```

   Saath me har chunk ka **source** (marker, file, chunk number, score, pehle 180 characters) alag list me rakha jata hai.
6. **GENERATE**: LLM ko 2 message jaate hain:
   - **System:** niyam, jaise "sirf CONTEXT se jawab do", "har claim ke aage [1] lagao", "jawab nahi mila to saaf bolo", "chhota jawab do".
   - **User:** `CONTEXT: ... SAWAL: P1 ticket ka SLA kitna hai?`

   Settings: `Temperature = 0.1` (creativity kam, factual jawab) aur `MaxOutputTokens = 800`.
7. **Jawab wapas**: `{ "answer": "...[1]", "sources": [...], "retrievedCount": 4 }`. UI jawab dikhata hai, aur neeche "Jawab in chunks se bana" box me sources dikhata hai.

> **Kyun sources dikhana zaroori hai?**
> LLM kabhi bhi galti kar sakta hai. Source dekhkar user khud verify kar sakta hai ki jawab document me sach me likha hai. Bina citation wale RAG par production me koi bharosa nahi karta.

# 7. Data flow 3: Agent aur Streaming

## 7.1 Agent mode (`POST /api/agent/ask`)

1. **Request**: `{ "question": "INC-1001 ka status batao aur uski priority ka SLA bhi", "sessionId": null }`
2. **Agent banta hai** (`SupportAgentFactory.CreateAgent()`): `ChatClientAgent` = LLM + instructions ("pehle search_documents chalao, apni yaadasht se mat bolo...") + 3 tools.
3. **Session** (`GetOrCreateSessionAsync`): `sessionId` mila aur dictionary me hai to purani memory wapas aati hai. Nahi to nayi session banti hai aur naya ID diya jata hai.
4. **Agent loop** (`agent.RunAsync(question, session)`). Ye loop library chalati hai. Neeche example hai; LLM ke exact shabd har baar alag ho sakte hain:

```
LLM ko milta hai: instructions + pichli baatein + sawaal + 3 tools ki list
  -> LLM: "get_ticket_status(ticketId: INC-1001) chalao"
     C#:  "Ticket INC-1001: status=In Progress, owner=Rahul Verma, priority=P2"
  -> LLM: "search_documents(query: P2 SLA) chalao"
     C#:  "--- itsm-policy.md (chunk 0, score 0.6xx) --- P2 (High): resolution SLA 8 ghante ..."
  -> LLM: final jawab likhta hai (ab koi tool nahi)
     "INC-1001 abhi In Progress hai (owner Rahul Verma). Priority P2 hai,
      jiska resolution SLA 8 ghante hai (itsm-policy.md)."
```

5. **Kaunse tool chale**: API response ke messages me se `FunctionCallContent` nikaal kar tool names ki list banati hai.
6. **Jawab**: `{ "answer": "...", "sessionId": "ab12...", "toolsUsed": ["get_ticket_status", "search_documents"] }`. UI `sessionId` yaad rakhta hai aur agle sawaal me bhejta hai. Tools chhote tags (chips) me dikhte hain.

> **Memory ka example**
> Pehla sawaal: "INC-1001 ka status batao". Doosra sawaal: **"aur uska owner kaun hai?"** Doosre sawaal me ticket number hai hi nahi. Lekin same `sessionId` ki wajah se agent ko pichli baat yaad hai, isliye wo samajh jata hai ki "uska" matlab INC-1001.

## 7.2 Agent + Streaming mode (`POST /api/agent/stream`)

Agent wala hi kaam hota hai, bas `RunStreamingAsync` se. Jawab tukdon me aata hai. Server ek hi HTTP response me ye events bhejta hai:

```
data: {"type":"session","sessionId":"ab12..."}
data: {"type":"tool","name":"get_ticket_status"}
data: {"type":"tool","name":"search_documents"}
data: {"type":"token","text":"INC"}
data: {"type":"token","text":"-1001 abhi"}
data: {"type":"token","text":" In Progress hai"}
...
data: {"type":"done"}
```

UI pehle ek khaali chat bubble banata hai. `tool` event aane par "Tool chal raha hai: get_ticket_status" dikhata hai, aur har `token` event par text bubble me jodta jata hai. Isse user ko jawab live banta hua dikhta hai.

## 7.3 Plain RAG vs Agent: farq

| | Plain RAG | Agent |
|---|---|---|
| Search kab hoti hai | Har sawaal par, hamesha | Jab agent ko zaroorat lage |
| Tools | Nahi | 3 tools (search, ticket status, create ticket) |
| Memory | Nahi, har sawaal alag | Haan, session ke through |
| Citations | Haan, numbered [1] [2] aur source box | File name jawab ke text me |
| LLM calls | 1 (seedha) | 2 ya zyada (har tool ke baad ek) |
| Speed | Tez | Dheema (zyada steps) |
| Kab use karein | Seedha document Q&A | Multi-step kaam, actions, follow-up sawaal |

# 8. Data kahan save hota hai

> **Sabse zaroori baat**
> Is project me **koi SQL database nahi hai**. "Database" ka matlab yahan sirf **vector store** hai. Abhi wo bhi **InMemory** hai, yaani sab kuch API process ki **RAM** me hai. **API restart karte hi saare documents, chat memory aur naye tickets gayab ho jaate hain.**

| Data | Kahan rehta hai | Code me kahan | API restart ke baad? |
|---|---|---|---|
| Uploaded file | **Kahin nahi.** Sirf text bheja jata hai, file save nahi hoti. | `app.ts` onFileSelected | - |
| Chunks + vectors | InMemory collection `askmydocs` (API ki RAM) | `Program.cs`, `IngestionService` | **Gayab** |
| Chunks + vectors (Qdrant mode) | Qdrant container, Docker volume `qdrant_data` (disk) | `docker-compose.yml` | Bache rehte hain |
| Documents ki list (Catalog) | static `ConcurrentDictionary` (RAM) | `IngestionService.Catalog` | **Gayab** (Qdrant mode me bhi!) |
| Chat memory (sessions) | `ConcurrentDictionary` (RAM) | `SupportAgentFactory._sessions` | **Gayab** |
| Tickets | static `Dictionary` me 3 demo tickets (RAM) | `SupportTools.Tickets` | Wapas 3 demo tickets; naye gayab |
| Chat screen (bubbles) | Browser ki memory (Angular signals) | `app.ts` turns | Page refresh par gayab |
| AI models | Ollama ka folder (`C:\Users\<aap>\.ollama\models`) | - | Rehte hain |
| Settings | `appsettings.json` | - | Rehte hain |

## Production me ye kaise hota

| Data | Abhi | Production me |
|---|---|---|
| Vectors | InMemory | Qdrant / Azure AI Search / pgvector |
| Documents list, tickets | Dictionary | SQL Server / PostgreSQL table |
| Chat sessions | Dictionary | Redis ya SQL (session ko JSON me serialize karke, Lab 6 me dikhaya hai) |
| Original files | Save nahi hoti | Blob storage (Azure Blob / S3) |

# 9. Semantic Kernel vs Agent Framework

## 9.1 Kernel kya karta hai?

Semantic Kernel (SK) Microsoft ki AI library hai. Iska center **Kernel** object hai, jo teen cheezein ek jagah jodta hai:

1. **AI service**: kaunsa LLM use karna hai (`AddOpenAIChatCompletion`).
2. **Plugins**: aapke C# functions jinke upar `[KernelFunction]` aur `[Description]` laga hai.
3. **Prompt functions**: prompt template ko hi ek function bana dena, jaise `"... {{$description}} ko {{$style}} style me summarise karo"`.

Jab aap `FunctionChoiceBehavior.Auto()` set karte ho, Kernel LLM ko plugins ki list bhejta hai. LLM khud tay karta hai ki kaunsa function chalana hai. Kernel wo function chalakar result LLM ko wapas deta hai.

```
// Lab05_SemanticKernel.cs (short me)
var builder = Kernel.CreateBuilder();
builder.AddOpenAIChatCompletion(AiConfig.ChatModel, AiConfig.CreateOpenAIClient());
builder.Plugins.AddFromObject(new ItsmPlugin(), "Itsm");   // GetTicketStatus, GetSlaHours
var kernel = builder.Build();

var settings = new OpenAIPromptExecutionSettings { FunctionChoiceBehavior =
FunctionChoiceBehavior.Auto() };
var reply = await chatService.GetChatMessageContentAsync(history, settings, kernel);
```

## 9.2 Main API me Semantic Kernel kyun nahi?

Lab 5 ke comments ke hisaab se Semantic Kernel ab **maintenance mode** me hai. Microsoft ki usi team ne **Microsoft Agent Framework** (`Microsoft.Agents.AI`) banaya hai, jo SK aur AutoGen ko milakar bana agla version hai. Isliye **naya code Agent Framework me likha jata hai**, aur main API wahi use karti hai.

| Semantic Kernel (Lab 5) | Agent Framework (main API, Lab 6, Lab 7) |
|---|---|
| `Kernel` | `ChatClientAgent` / `AIAgent` |
| Plugin + `[KernelFunction]` | `AIFunctionFactory.Create(method)` → tools |
| `ChatHistory` | `AgentSession` |
| `FunctionChoiceBehavior.Auto()` | `tools:` parameter + `UseFunctionInvocation()` |
| - | Multi-agent workflows (`AgentWorkflowBuilder`) |

> **To SK kyun seekhein?**
> Interview me abhi bhi poocha jata hai, bahut saara purana production code SK me hai, aur concepts (plugin/tool, function calling, prompt template) dono me same hain. Bas naam badle hain.

# 10. Har module ki summary

## 10.1 API ( `api/AskMyDocs.Api` )

| File | Kya karti hai |
|---|---|
| `Program.cs` | App ka start point. Config padhti hai, LLM aur embedding clients banati hai, vector store chunti hai (InMemory/Qdrant), services register karti hai, CORS set karti hai, aur saare endpoints banati hai. |
| `Models/Contracts.cs` | Request/response ke shapes: `IngestRequest`, `IngestResponse`, `AskRequest`, `AskResponse`, `SourceRef`, `AgentAskResponse`, `DocumentSummary`. |
| `Models/DocChunk.cs` | Vector store ka "table structure": ek chunk me kya-kya save hota hai. Vector size 768. |
| `Rag/TextChunker.cs` | Document ko 1200-character chunks me todta hai (200 overlap). |
| `Rag/IngestionService.cs` | Upload ka poora kaam: chunk karo, records banao, vector store me save karo, catalog me entry daalo. |
| `Rag/RagService.cs` | Sawaal ka jawab: retrieve (top 4, score ≥ 0.25), augment (context banao), generate (LLM). `RetrieveAsync` agent ka search tool bhi use karta hai. |
| `Agents/SupportTools.cs` | Agent ke 3 tools: `search_documents`, `get_ticket_status`, `create_ticket`. Tickets demo data hain. |
| `Agents/SupportAgentFactory.cs` | Agent banata hai (instructions + tools) aur sessions (memory) sambhalta hai. |
| `appsettings.json` | AI endpoint, model names, API key, vector store type. |
| `Properties/launchSettings.json` | Port 5080 (http) / 7080 (https), environment = Development. |
| `AskMyDocs.Api.http` | Ready-made test requests. VS / VS Code se bina UI ke API test karne ke liye. |

## 10.2 Angular UI ( `web/askmydocs-web/src` )

| File | Kya karti hai |
|---|---|
| `main.ts` | Angular app start karta hai ( `bootstrapApplication` ). |
| `app/app.config.ts` | HttpClient setup ( `withFetch()` ) aur global error listeners. |
| `app/app.ts` | Poori screen ka logic: documents list, chat turns, mode, upload, send, streaming handle karna. Saari state **signals** me hai. |
| `app/app.html` | Screen ka layout: left sidebar (upload, documents, mode) aur right chat area (bubbles, tools, sources, input box). |
| `app/app.css` | Styling. |
| `app/askmydocs.service.ts` | API se baat karne wali service: upload, list, ask, askAgent, streamAgent (SSE). API ka address `http://localhost:5080` yahin likha hai. |
| `app/models.ts` | TypeScript interfaces. C# ke `Contracts.cs` se exact match karte hain. |

## 10.3 Labs ( `labs/AskMyDocs.Labs` )

| Lab | Kya sikhata hai | AI chahiye? |
|---|---|---|
| `Program.cs` | Dispatcher: `dotnet run -- labN` se sahi lab chalata hai, error aane par aam wajah batata hai. | - |
| `AiConfig.cs` | Saare labs ke liye LLM/embedding client banata hai. Cosine similarity ka formula haath se likha hai. | - |
| Lab 1: Embeddings | Text ka vector dekho. 4 sentences ko ek query se compare karke similarity score dekho. | Haan |
| Lab 2: Vector Store | 5 FAQ save karo, semantic search karo, category filter ke saath search karo. | Haan |
| Lab 3: Chunking | Ek leave policy ko 3 alag size me todkar farq dekho. | **Nahi** |
| Lab 4: Mini RAG | Same sawaal bina RAG aur RAG ke saath poochkar farq dekho. | Haan |
| Lab 5: Semantic Kernel | Prompt function (template) aur auto function calling (plugin). | Haan |
| Lab 6: Agent | Simple agent, tools wala agent, session (memory) aur use JSON me save karna. | Haan |
| Lab 7: Multi-agent | 3 agents ki pipeline: Triage → Solver → Editor. | Haan |

## 11. Common errors aur unka hal

Ye wahi errors hain jo setup ke time aaye the, aur kuch aur jo aa sakte hain.

| Error | Wajah | Hal |
|---|---|---|
| `Value cannot be an empty string. (Parameter 'key')` (API start par crash) | `ApiKey` khali string tha aur code sirf `null` check karta tha. | **Fix ho chuka hai** (`Program.cs`). Ab khali key par placeholder lagta hai. |
| `Upload fail: ... 500 Internal Server Error` + andar `401 invalid_api_key` | OpenAI endpoint par bina key ke request gayi. | Ollama par shift kar diya. Ya OpenAI key daalo. |
| `Cannot find path '...\api\AskMyDocs.Api\api\AskMyDocs.Api'` | Aap pehle se us folder me the, aur `cd` ne andar ek aur folder dhoondha. | Poora path use karo: `cd C:\Users\...\AskMyDocs\api\AskMyDocs.Api` |
| `Couldn't find a project to run` | `dotnet run` Angular folder me chalaya. | Angular me `npm start` chalta hai. `dotnet run` API folder me. |
| `MSB3027 ... file is locked by AskMyDocs.Api` | Purani API abhi bhi kisi doosre terminal me chal rahi hai. | Us terminal me Ctrl+C dabao. Ya Task Manager se `AskMyDocs.Api.exe` band karo. |
| Dimension mismatch | Vector size (768/1536) aur embedding model match nahi karte. | `nomic-embed-text` = 768, `text-embedding-3-small` = 1536. `DocChunk.cs`, Lab02, Lab04 me sahi number rakho. |
| UI: "API se connect nahi ho pa raha" | API band hai, ya 5080 ke alawa kisi port par hai. | Pehle API chalao, phir page refresh karo. |
| Connection refused `localhost:11434` | Ollama band hai. | Start menu se Ollama chalao, ya terminal me `ollama serve`. |
| Jawab bahut dheere aata hai | Machine me GPU nahi hai, model CPU par chalta hai. | Normal hai. Chhota model (`llama3.2`) isi wajah se liya hai. |

| | | |
|---|---|---|
| Agent kabhi tool sahi nahi chunta | Chhota local model (3B) tool calling me `gpt-4o-mini` jitna achha nahi hai. | Sawaal saaf likho. Ya bada model / OpenAI use karo. |
| Angular analytics wala sawaal | Angular CLI ek baar poochta hai. | Yes/No kuch bhi chuno, app par asar nahi padta. |

## 12. Seekhne ka sahi order

1. **Lab 3 (Chunking)**: bina AI ke chalta hai. Samjho ki document kaise tootta hai.
2. **Lab 1 (Embeddings)**: text numbers kaise banta hai, aur similarity kaise naapte hain.
3. **Lab 2 (Vector Store)**: save, search aur filter.
4. **Lab 4 (Mini RAG)**: bina RAG vs RAG. Yahan "aha" moment aata hai.
5. **API code padho**: `Program.cs` → `TextChunker.cs` → `IngestionService.cs` → `RagService.cs`. Ab ye sab Lab 1-4 ka hi bada version lagega.
6. **Lab 6 (Agent)**: agent, tools, session.
7. **API ke Agents folder**: `SupportTools.cs` → `SupportAgentFactory.cs`, phir `Program.cs` ke agent endpoints.
8. **Angular**: `askmydocs.service.ts` (khaas kar `streamAgent`) → `app.ts` → `app.html`.
9. **Lab 7 (Multi-agent)** aur **Lab 5 (Semantic Kernel)**: aage ke liye.
10. **Khud kuch badlo**: chunk size (1200 → 600), TopK (4 → 2), MinScore, system prompt. Har badlav ke baad same sawaal poochkar farq dekho.

**Practice ke liye chhote tasks**

- `GET /api/documents/{id}/chunks` endpoint banao jo kisi document ke saare chunks dikhaye.
- `DELETE /api/documents/{id}` banao (`DocumentId` filter se chunks dhoondhkar delete karo).
- Ek naya agent tool banao, jaise `get_sla_hours(priority)`.
- Qdrant chalakar dekho ki API restart ke baad bhi search kaam karta hai.